

Whitespaces Wording Revamp

Document #: P2348R0
Date: 2021-04-13
Project: Programming Language C++
Audience: SG-16, CWG
Reply-to: Corentin Jabot <corentin.jabot@gmail.com>

Design

This paper aims to clarify what constitutes a new-line and a whitespace, using Unicode terminology (which is consistent with [P2314R1](#) [2]). No breaking behavior change is intended.

This paper resolves [CWG2002](#) [4] and [CWG1655](#) [3].

Make whitespace grammar elements

For clarity, this paper introduces a grammar for whitespaces, including comments, and then refers to whitespace as grammar terms. This also makes it easier to later extend the list of whitespaces. However, the questions of extending the list of Unicode characters treated as whitespace is not explored in this paper. Non-Unicode source encoding would continue to map their set of whitespaces and new-line to what is currently designated in the standard: CR, FF, LF, VT, SPACE, TAB.

Comments

The proposed wording change makes vertical tabs end // comments, rather than make them ill-formed no diagnostic required. This is consistent with Unicode which considers vertical tab and form feed to be line breaks. An alternative would be to make vertical tab/form feed in comments ill-formed. But this would not be consistent with Unicode guidelines.

Replace the term *new-line* by *line-break*

`new-line` in the wording refers to both an unspecified character to which all source line terminator map to and a specific, implementation-defined character in the literal character set. Using different terminology makes it clearer which is which.

Do not perform whitespace replacement

All whitespaces (including comments) are conserved by the proposed wording through phases 1-7. In practice, this is not observable (and we may want a note stating that somewhere). This fixes [CWG2002](#) [4].

According to Clause 15 [cpp] paragraph 4,

The only white-space characters that shall appear between preprocessing tokens within a preprocessing directive (from just after the introducing # preprocessing token through just before the terminating new-line character) are space and horizontal-tab (including spaces that have replaced comments or possibly other white-space characters in translation phase 3).

The effect of this restriction is unclear, however, since translation phase 3 is permitted to transform all white space characters and comments into spaces. The relationship between these two rules should be clarified.

The list of whitespaces is sufficient to address the needs of codepoints conservations through phases 1-6 of [P2295R2](#) [?]. The clause mentioned in CWG2002 is deleted.

The set of character is not expended

We do not propose new characters as whitespaces. However, following Unicode guidelines, we clarify that CRLF is to be considered a single line break.

Raw-string literals

For clarity, that line-breaks are mapped to a new-line in raw-string-literal is made normative. We intend that this paper addresses [CWG1655](#) [3]: by using different terms for *line-break* and *new-line*, we make clear that these things are not related.

Wording



Phases of translation

[lex.phases]

The precedence among the syntax rules of translation is specified by the following phases.

1. Physical source file characters are mapped, in an implementation-defined manner, to the basic source character set (~~introducing new-line characters for end-of-line indicators~~) if necessary. The set of physical source file characters accepted is implementation-defined. Any source file character not in the basic source character set is replaced by the *universal-character-name* that designates that character. An implementation may use any internal encoding, so long as an actual extended character encountered in the source file, and the same extended character expressed in the source file as a *universal-character-name* (e.g., using the `\uXXXX` notation), are handled equivalently except where this replacement is reverted in a raw string literal.
2. Each instance of a backslash character (`\`) immediately followed by a ~~new-line character~~ *line-break* is deleted, splicing physical source lines to form logical source lines. Only the

last backslash on any physical source line shall be eligible for being part of such a splice. Except for splices reverted in a raw string literal, if a splice results in a character sequence that matches the syntax of a *universal-character-name*, the behavior is undefined. A source file that is not empty and that does not end in a ~~new-line-character~~ *line-break*, or that ends in a ~~new-line-character~~ *line-break* immediately preceded by a backslash character before any such splicing takes place, shall be processed as if an additional ~~new-line-character~~ *line-break* were appended to the file.

3. The source file is decomposed into preprocessing tokens and ~~sequences-of-whitespace characters (including comments)~~ *whitespace*. A source file shall not end in a partial preprocessing token or in a partial comment. ~~Each comment is replaced by one space character. New-line characters are~~ *Each line-break is* retained. ~~Whether each nonempty sequence of whitespace characters other than new-line is retained or replaced by one space character is unspecified.~~ The process of dividing a source file's characters into preprocessing tokens is context-dependent. [*Example*: See the handling of < within a #include preprocessing directive. — *end example*]
4. Preprocessing directives are executed, macro invocations are expanded, and _Pragma unary operator expressions are executed. If a character sequence that matches the syntax of a *universal-character-name* is produced by token concatenation, the behavior is undefined. A #include preprocessing directive causes the named header or source file to be processed from phase 1 through phase 4, recursively. All preprocessing directives are then deleted.
5. Each *basic-c-char*, *basic-s-char*, and *r-char* in a *character-literal* or a *string-literal*, as well as each *escape-sequence* and *universal-character-name* in a *character-literal* or a non-raw string literal, is encoded in the literal's associated character encoding as specified in ?? and ??.
6. Adjacent *string-literal* s are concatenated and a null character is appended to the result as specified in ??.
7. ~~Whitespace characters~~ *whitespace* separating tokens are no longer significant. Each preprocessing token is converted into a token. The resulting tokens are syntactically and semantically analyzed and translated as a translation unit. [*Note*: The process of analyzing and translating the tokens can occasionally result in one token being replaced by a sequence of other tokens. — *end note*] It is implementation-defined whether the sources for module units and header units on which the current translation unit has an interface dependency (??, ??) are required to be available. [*Note*: Source files, translation units and translated translation units need not necessarily be stored as files, nor need there be any one-to-one correspondence between these entities and any external representation. The description is conceptual only, and does not specify any particular implementation. — *end note*]

❖ Preprocessing tokens

[lex.pptoken]

preprocessing-token:
 header-name
 import-keyword
 module-keyword
 export-keyword
 identifier
 pp-number
 character-literal
 user-defined-character-literal
 string-literal
 user-defined-string-literal
 preprocessing-op-or-punc
 each non-whitespace character that cannot be one of the above

Each preprocessing token that is converted to a token shall have the lexical form of a keyword, an identifier, a literal, or an operator or punctuation.

A preprocessing token is the minimal lexical element of the language in translation phases 3 through 6. The categories of preprocessing token are: header names, placeholder tokens produced by preprocessing `import` and `module` directives (*import-keyword*, *module-keyword*, and *export-keyword*), identifiers, preprocessing numbers, character literals (including user-defined character literals), string literals (including user-defined string literals), preprocessing operators and punctuators, and single non-whitespace characters that do not lexically match the other preprocessing token categories. If a ' or a " character matches the last category, the behavior is undefined. Preprocessing tokens can be separated by ~~whitespace; whitespace . this consists of comments, or whitespace characters (space, horizontal tab, new-line, vertical tab, and form-feed), or both.~~ As described in [cpp], in certain circumstances during translation phase 4, ~~whitespace~~ whitespace (or the absence thereof) serves as more than preprocessing token separation. ~~Whitespace~~ whitespace can appear within a preprocessing token only as part of a header name or between the quotation characters in a character literal or string literal.

❖ Tokens

[lex.token]

token:
 identifier
 keyword
 literal
 operator-or-punctuator

~~There are five kinds of tokens: identifiers, keywords, literals, operators, and other separators.~~

Blanks, horizontal and vertical tabs, newlines, formfeeds, and comments (collectively, "whitespace"), as described below, are ignored except as they serve to separate tokens. [Note: Some ~~whitespace~~ whitespace is required to separate otherwise adjacent identifiers, keywords, numeric literals, and alternative tokens containing alphabetic characters. — end note]



Whitespaces

[lex.whitespaces]

whitespace:

horizontal-whitespace
line-break

horizontal-whitespace:

horizontal-whitespace
comment
horizontal-whitespace-character

comment:

// any member of the basic source character set except *line-break*
*/** any member of the basic source character set **/*

line-break:

U+000A LINE FEED
U+000B VERTICAL TAB
U+000C FORM FEED
U+000D CARRIAGE RETURN
U+000D CARRIAGE RETURN followed by U+000A LINE FEED

horizontal-whitespace-character:

U+0009 HORIZONTAL TAB
U+0020 SPACE



Comments

[lex.comment]

~~The characters */** start a comment, which terminates with the characters **/*. These c~~omments do not nest. ~~The characters *//* start a comment, which terminates immediately before the next new-line character. If there is a form-feed or a vertical-tab character in such a comment, only whitespace characters shall appear between it and the new-line that terminates the comment; no diagnostic is required.~~ [Note: The comment characters *//*, */**, and **/* have no special meaning within a *//* comment and are treated just like other characters. Similarly, the comment characters *//* and */** have no special meaning within a */** comment. — end note]

whitespaces are ignored except as they serve to separate tokens.



Header names

[lex.header]

header-name:

< h-char-sequence >
" q-char-sequence "

h-char-sequence:

h-char
h-char-sequence h-char

h-char:

any member of the source character set except **new-line** line-break and >

q-char-sequence:

q-char

q-char-sequence q-char

q-char:

any member of the source character set except **new-line** line-break and "

[*Note:* Header name preprocessing tokens only appear within a #include preprocessing directive, a __has_include preprocessing expression, or after certain occurrences of an import token (see ??). — *end note*] The sequences in both forms of *header-names* are mapped in an implementation-defined manner to headers or to external source file names as specified in ??.

The appearance of either of the characters ' or \ or of either of the character sequences /* or // in a *q-char-sequence* or an *h-char-sequence* is conditionally-supported with implementation-defined semantics, as is the appearance of the character " in an *h-char-sequence*.



Character literals

[lex.ccon]

character-literal:

opt **encoding-prefix** ' *c-char-sequence* '

encoding-prefix: one of

u8 u U L

c-char-sequence:

c-char

c-char-sequence c-char

c-char:

basic-c-char

escape-sequence

universal-character-name

basic-c-char:

any member of the basic source character set except the single-quote ', backslash

\, or **new-line character** line-break

escape-sequence:

simple-escape-sequence

numeric-escape-sequence

conditional-escape-sequence

simple-escape-sequence:

\ *simple-escape-sequence-char*

simple-escape-sequence-char: one of

' " ? \ a b f n r t v

numeric-escape-sequence:

octal-escape-sequence

hexadecimal-escape-sequence

octal-escape-sequence:

\ octal-digit
\ octal-digit octal-digit
\ octal-digit octal-digit octal-digit

hexadecimal-escape-sequence:

\x hexadecimal-digit
hexadecimal-escape-sequence hexadecimal-digit

conditional-escape-sequence:

\ conditional-escape-sequence-char

conditional-escape-sequence-char:

any member of the basic source character set that is not an *octal-digit*, a *simple-escape-sequence-char*, or the characters *u*, *U*, or *x*

//...

The character specified by a *simple-escape-sequence* is specified in . [*Note:* Using an escape sequence for a question mark is supported for compatibility with ISO C++ 2014 and ISO C. — *end note*]

Table 1: Simple escape sequences

new-line	NL(LF)	<code>\n</code>
horizontal tab	HT	<code>\t</code>
vertical tab	VT	<code>\v</code>
backspace	BS	<code>\b</code>
carriage return	CR	<code>\r</code>
form feed	FF	<code>\f</code>
alert	BEL	<code>\a</code>
backslash	<code>\</code>	<code>\\</code>
question mark	<code>?</code>	<code>\?</code>
single quote	<code>'</code>	<code>\'</code>
double quote	<code>"</code>	<code>\"</code>



String literals

[lex.string]

string-literal:

^{opt}encoding-prefix " ^{opt}s-char-sequence "
^{opt}encoding-prefix R raw-string

s-char-sequence:

s-char
s-char-sequence s-char

s-char:

basic-s-char
escape-sequence
universal-character-name

basic-s-char:

any member of the translation character set except the double-quote `"`, backslash `\`, or **new-line character** [line-break](#)

raw-string:

" *opt**d-char-sequence* (*opt**r-char-sequence*) *opt**d-char-sequence* "

r-char-sequence:

r-char

r-char-sequence *r-char*

r-char:

any member of the source character set, except a right parenthesis) followed by the initial *d-char-sequence* (which may be empty) followed by a double quote ".

d-char-sequence:

d-char

d-char-sequence *d-char*

d-char:

any member of the basic source character set except:

space *horizontal-whitespace-character*, *line-break*, the left parenthesis (, the right parenthesis), the backslash \, ~~and the control characters representing horizontal tab, vertical tab, form feed, and newline.~~

A *string-literal* that has an R in the prefix is a *raw string literal*. The *d-char-sequence* serves as a delimiter. The terminating *d-char-sequence* of a *raw-string* is the same sequence of characters as the initial *d-char-sequence*. A *d-char-sequence* shall consist of at most 16 characters.

[*Note*: The characters ' (' and ') ' are permitted in a *raw-string*. Thus, R"delimiter((a|b))delimiter" is equivalent to "(a|b)". — *end note*]

[*Note*: A ~~source-file new-line~~ *line-break* in a raw string literal results in a new-line in the resulting execution string literal. Assuming no whitespace at the beginning of lines in the following example, the assert will succeed:

```
const char* p = R"(a\
b
c)";
assert(std::strcmp(p, "a\\nb\\nc") == 0);
```

— *end note*]

[*Example*: The raw string

```
R"a(
)\
a"
)a"
```

is equivalent to "\n)\\na\"\\n". The raw string

```
R"(x = "\"y\"")"
```

is equivalent to "x = \"\\\"y\\\"\"". — *end example*]

[cpp]



Preamble

[cpp.pre]

preprocessing-file:

*opt*group
module-file

module-file:

*opt*pp-global-module-fragment pp-module *opt*group *opt*pp-private-module-fragment

pp-global-module-fragment:

module ; **new-line** line-break *opt*group

pp-private-module-fragment:

module : private ; **new-line** line-break *opt*group

group:

group-part
group group-part

group-part:

control-line
if-section
text-line
conditionally-supported-directive

control-line:

include pp-tokens **new-line** line-break
pp-import
define identifier replacement-list **new-line** line-break
define identifier lparen *opt*identifier-list) replacement-list **new-line** line-break
define identifier lparen ...) replacement-list **new-line** line-break
define identifier lparen identifier-list , ...) replacement-list **new-line** line-break
undef identifier **new-line** line-break
line pp-tokens **new-line** line-break
error *opt*pp-tokens **new-line** line-break
pragma *opt*pp-tokens **new-line** line-break
new-line line-break

if-section:

if-group *opt*elif-groups *opt*else-group endif-line

if-group:

if constant-expression **new-line** line-break *opt*group
ifdef identifier **new-line** line-break *opt*group
ifndef identifier **new-line** line-break *opt*group

elif-groups:

elif-group
elif-groups elif-group

elif-group:

elif constant-expression **new-line** line-break *opt*group

else-group:
 # *else* *new-line* *line-break* *opt**group*
endif-line:
 # *endif* *new-line* *line-break*
text-line:
 *opt**pp-tokens* *new-line* *line-break*
conditionally-supported-directive:
 pp-tokens *new-line* *line-break*
lparen:
 a (character not immediately preceded by *whitespace* *whitespace*
identifier-list:
 identifier
 identifier-list , *identifier*
replacement-list:
 *opt**pp-tokens*
pp-tokens:
 preprocessing-token
 pp-tokens *preprocessing-token*
new-line:
 the *new-line* character

A *preprocessing directive* consists of a sequence of preprocessing tokens that satisfies the following constraints: At the start of translation phase 4, the first token in the sequence, referred to as a *directive-introducing token*, begins with the first *character in the source file (optionally after whitespace containing no new-line characters)* *non-whitespace* or follows *whitespace* *whitespace* containing at least one *new-line-character* *line-break*, and is

- a # preprocessing token, or
- an *import* preprocessing token immediately followed on the same logical line by a *header-name*, <, *identifier*, *string-literal*, or : preprocessing token, or
- a *module* preprocessing token immediately followed on the same logical line by an *identifier*, :, or ; preprocessing token, or
- an *export* preprocessing token immediately followed on the same logical line by one of the two preceding forms.

The last token in the sequence is the first token within the sequence that is immediately followed by *whitespace* *whitespace* containing a *new-line-character* *line-break*. [Note: Thus, preprocessing directives are commonly called “lines”. These “lines” have no other syntactic significance, as all *whitespace* *whitespace* is equivalent except in certain situations during preprocessing (see the # character string literal creation operator in ??, for example). — end note] [Note: A *new-line-character* *line-break* ends the preprocessing directive even if it occurs within what would otherwise be an invocation of a function-like macro. — end note]

[Example:

```

#                // preprocessing directive
module ;         // preprocessing directive
export module leftpad; // preprocessing directive
import <string>;  // preprocessing directive
export import "squee"; // preprocessing directive
import rightpad;  // preprocessing directive
import :part;     // preprocessing directive

module          // not a preprocessing directive
;              // not a preprocessing directive

export         // not a preprocessing directive
import        // not a preprocessing directive
foo;          // not a preprocessing directive

export        // not a preprocessing directive
import foo;   // preprocessing directive (ill-formed at phase 7)

import ::     // not a preprocessing directive
import ->     // not a preprocessing directive

```

— *end example*]

A sequence of preprocessing tokens is only a *text-line* if it does not begin with a directive-introducing token. A sequence of preprocessing tokens is only a *conditionally-supported-directive* if it does not begin with any of the directive names appearing after a # in the syntax. A *conditionally-supported-directive* is conditionally-supported with implementation-defined semantics.

At the start of phase 4 of translation, the *group* of a *pp-global-module-fragment* shall contain neither a *text-line* nor a *pp-import*.

When in a group that is skipped, the directive syntax is relaxed to allow any sequence of preprocessing tokens to occur between the directive name and the following ~~new-line character~~ line-break .

The only whitespace characters that shall appear between preprocessing tokens within a preprocessing directive (from just after the directive-introducing token through just before the terminating new-line character) are space and horizontal-tab (including spaces that have replaced comments or possibly other whitespace characters in translation phase 3).

The implementation can process and skip sections of source files conditionally, include other source files, import macros from header units, and replace macros. These capabilities are called *preprocessing*, because conceptually they occur before translation of the resulting translation unit.

The preprocessing tokens within a preprocessing directive are not subject to macro expansion unless otherwise stated.

[*Example*: In:

```
#define EMPTY
EMPTY # include <file.h>
```

the sequence of preprocessing tokens on the second line is *not* a preprocessing directive, because it does not begin with a # at the start of translation phase 4, even though it will do so after the macro `EMPTY` has been replaced. — *end example*]



Conditional inclusion

[cpp.cond]

defined-macro-expression:

```
defined identifier
defined ( identifier )
```

h-preprocessing-token:

```
any preprocessing-token other than >
```

h-pp-tokens:

```
h-preprocessing-token
h-pp-tokens h-preprocessing-token
```

header-name-tokens:

```
string-literal
< h-pp-tokens >
```

has-include-expression:

```
__has_include ( header-name )
__has_include ( header-name-tokens )
```

has-attribute-expression:

```
__has_cpp_attribute ( pp-tokens )
```

The expression that controls conditional inclusion shall be an integral constant expression except that identifiers (including those lexically identical to keywords) are interpreted as described below ¹ because the controlling constant expression is evaluated during translation phase 4, all identifiers either are or are not macro names — there simply are no keywords, enumeration constants, etc. and it may contain zero or more *defined-macro-expressions* and/or *has-include-expressions* and/or *has-attribute-expressions* as unary operator expressions.

A *defined-macro-expression* evaluates to 1 if the identifier is currently defined as a macro name (that is, if it is predefined or if it has one or more active macro definitions, for example because it has been the subject of a `#define` preprocessing directive without an intervening `#undef` directive with the same subject identifier), 0 if it is not.

The second form of *has-include-expression* is considered only if the first form does not match, in which case the preprocessing tokens are processed just as in normal text.

The header or source file identified by the parenthesized preprocessing token sequence in each contained *has-include-expression* is searched for as if that preprocessing token sequence were the *pp-tokens* in a `#include` directive, except that no further macro expansion is performed.

¹B

If such a directive would not satisfy the syntactic requirements of a `#include` directive, the program is ill-formed. The *has-include-expression* evaluates to 1 if the search for the source file succeeds, and to 0 if the search fails.

Each *has-attribute-expression* is replaced by a non-zero *pp-number* matching the form of an *integer-literal* if the implementation supports an attribute with the name specified by interpreting the *pp-tokens*, after macro expansion, as an *attribute-token*, and by 0 otherwise. The program is ill-formed if the *pp-tokens* do not match the form of an *attribute-token*.

For an attribute specified in this document, the value of the *has-attribute-expression* is given by . For other attributes recognized by the implementation, the value is implementation-defined. [*Note*: It is expected that the availability of an attribute can be detected by any non-zero result. — *end note*]

Table 2: `__has_cpp_attribute` values

Attribute	Value
<code>carries_dependency</code>	200809L
<code>deprecated</code>	201309L
<code>fallthrough</code>	201603L
<code>likely</code>	201803L
<code>maybe_unused</code>	201603L
<code>no_unique_address</code>	201803L
<code>nodiscard</code>	201907L
<code>noreturn</code>	200809L
<code>unlikely</code>	201803L

The `#ifdef` and `#ifndef` directives, and the defined conditional inclusion operator, shall treat `__has_include` and `__has_cpp_attribute` as if they were the names of defined macros. The identifiers `__has_include` and `__has_cpp_attribute` shall not appear in any context not mentioned in this subclause.

Each preprocessing token that remains (in the list of preprocessing tokens that will become the controlling expression) after all macro replacements have occurred shall be in the lexical form of a token.

Preprocessing directives of the forms

```
# if      constant-expression new-line line-break optgroup
# elif   constant-expression new-line line-break optgroup
```

check whether the controlling constant expression evaluates to nonzero.

Prior to evaluation, macro invocations in the list of preprocessing tokens that will become the controlling constant expression are replaced (except for those macro names modified by the defined unary operator), just as in normal text. If the token defined is generated as a result of this replacement process or use of the defined unary operator does not match one of the two specified forms prior to macro replacement, the behavior is undefined.

After all replacements due to macro expansion and evaluations of *defined-macro-expression* s, *has-include-expression* s, and *has-attribute-expression* s have been performed, all remaining identifiers and keywords, except for `true` and `false`, are replaced with the *pp-number* 0, and then each preprocessing token is converted into a token. [*Note*: An alternative token is not

an identifier, even when its spelling consists entirely of letters and underscores. Therefore it is not subject to this replacement. — *end note*]

The resulting tokens comprise the controlling constant expression which is evaluated according to the rules of ?? using arithmetic that has at least the ranges specified in ?. For the purposes of this token conversion and evaluation all signed and unsigned integer types act as if they have the same representation as, respectively, `intmax_t` or `uintmax_t`. [*Note*: Thus on an implementation where `std::numeric_limits<int>::max()` is `0x7FFF` and `std::numeric_limits<unsigned int>::max()` is `0xFFFF`, the integer literal `0x8000` is signed and positive within a `#if` expression even though it is unsigned in translation phase 7. — *end note*] This includes interpreting *character-literal*s, which may involve converting escape sequences into execution character set members. Whether the numeric value for these *character-literal*s matches the value obtained when an identical *character-literal* occurs in an expression (other than within a `#if` or `#elif` directive) is implementation-defined. [*Note*: Thus, the constant expression in the following `#if` directive and `if` statement is not guaranteed to evaluate to the same value in these two contexts:

```
#if 'z' - 'a' == 25
if ('z' - 'a' == 25)
```

— *end note*] Also, whether a single-character *character-literal* may have a negative value is implementation-defined. Each subexpression with type `bool` is subjected to integral promotion before processing continues.

Preprocessing directives of the forms

```
# ifdef identifier new-line line-break optgroup
# ifndef identifier new-line line-break optgroup
```

check whether the identifier is or is not currently defined as a macro name. Their conditions are equivalent to `#if defined identifier` and `#if !defined identifier` respectively.

Each directive's condition is checked in order. If it evaluates to false (zero), the group that it controls is skipped: directives are processed only through the name that determines the directive in order to keep track of the level of nested conditionals; the rest of the directives' preprocessing tokens are ignored, as are the other preprocessing tokens in the group. Only the first group whose control condition evaluates to true (nonzero) is processed; any following groups are skipped and their controlling directives are processed as if they were in a group that is skipped. If none of the conditions evaluates to true, and there is a `#else` directive, the group controlled by the `#else` is processed; lacking a `#else` directive, all the groups until the `#endif` are skipped.²s indicated by the syntax, a preprocessing token cannot follow a `#else` or `#endif` directive before the terminating *new-line-character* *line-break*. However, comments can appear anywhere in a source file, including within a preprocessing directive.

[*Example*: This demonstrates a way to include a library optional facility only if it is available:

```
#if __has_include(<optional>)
# include <optional>
# if __cpp_lib_optional >= 201603
#   define have_optional 1
```

²A

```

# endif
#elif __has_include(<experimental/optional>)
# include <experimental/optional>
# if __cpp_lib_experimental_optional >= 201411
#     define have_optional 1
#     define experimental_optional 1
# endif
#endif
#ifndef have_optional
# define have_optional 0
#endif

```

— *end example*]

[*Example:* This demonstrates a way to use the attribute `[[acme::deprecated]]` only if it is available.

```

#if __has_cpp_attribute(acme::deprecated)
# define ATTR_DEPRECATED(msg) [[acme::deprecated(msg)]]
#else
# define ATTR_DEPRECATED(msg) [[deprecated(msg)]]
#endif
ATTR_DEPRECATED("This function is deprecated") void anvil();

```

— *end example*]



Source file inclusion

[cpp.include]

A `#include` directive shall identify a header or source file that can be processed by the implementation.

A preprocessing directive of the form `# include <h-char-sequence>` *new-line line-break* searches a sequence of implementation-defined places for a header identified uniquely by the specified sequence between the `<` and `>` delimiters, and causes the replacement of that directive by the entire contents of the header. How the places are specified or the header identified is implementation-defined.

A preprocessing directive of the form `# include "q-char-sequence"` *new-line line-break* causes the replacement of that directive by the entire contents of the source file identified by the specified sequence between the `"` delimiters. The named source file is searched for in an implementation-defined manner. If this search is not supported, or if the search fails, the directive is reprocessed as if it read `# include <h-char-sequence>` *new-line line-break* with the identical contained sequence (including `>` characters, if any) from the original directive.

A preprocessing directive of the form `# include pp-tokens` *new-line line-break* (that does not match one of the two previous forms) is permitted. The preprocessing tokens after `include` in the directive are processed just as in normal text (i.e., each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens). If the

directive resulting after all replacements does not match one of the two previous forms, the behavior is undefined. ³ote that adjacent *string-literal* s are not concatenated into a single *string-literal* (see the translation phases in ??); thus, an expansion that results in two *string-literal* s is an invalid directive. The method by which a sequence of preprocessing tokens between a < and a > preprocessing token pair or a pair of " characters is combined into a single header name preprocessing token is implementation-defined.

The implementation shall provide unique mappings for sequences consisting of one or more *nondigits* or *digits* followed by a period (.) and a single *nondigit*. The first character shall not be a *digit*. The implementation may ignore distinctions of alphabetical case.

A #include preprocessing directive may appear in a source file that has been read because of a #include directive in another file, up to an implementation-defined nesting limit.

If the header identified by the *header-name* denotes an importable header, it is implementation-defined whether the #include preprocessing directive is instead replaced by an import directive of the form `import header-name ;` *new-line* *line-break*

[*Note*: An implementation can provide a mechanism for making arbitrary source files available to the < > search. However, using the < > form for headers provided with the implementation and the " " form for sources outside the control of the implementation achieves wider portability. For instance:

```
#include <stdio.h>
#include <unistd.h>
#include "usefullib.h"
#include "myprog.h"
```

— *end note*]

[*Example*: This illustrates macro-replaced #include directives:

```
#if VERSION == 1
#define INCFILE "vers1.h"
#elif VERSION == 2
#define INCFILE "vers2.h" // and so on
#else
#define INCFILE "versN.h"
#endif
#include INCFILE
```

— *end example*]



Module directive

[cpp.module]

pp-module:

```
optexport module optpp-tokens ; new-line line-break
```

³N

A *pp-module* shall not appear in a context where `module` or (if it is the first token of the *pp-module*) `export` is an identifier defined as an object-like macro.

Any preprocessing tokens after the `module` preprocessing token in the `module` directive are processed just as in normal text. [*Note*: Each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens. — *end note*]

The `module` and `export` (if it exists) preprocessing tokens are replaced by the *module-keyword* and *export-keyword* preprocessing tokens respectively. [*Note*: This makes the line no longer a directive so it is not removed at the end of phase 4. — *end note*]



Header unit importation

[cpp.import]

pp-import:

```
optexport import header-name optpp-tokens ; new-line line-break  
optexport import header-name-tokens optpp-tokens ; new-line line-break  
optexport import pp-tokens ; new-line line-break
```

A *pp-import* shall not appear in a context where `import` or (if it is the first token of the *pp-import*) `export` is an identifier defined as an object-like macro.

The preprocessing tokens after the `import` preprocessing token in the `import` *control-line* are processed just as in normal text (i.e., each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens). [*Note*: An `import` directive matching the first two forms of a *pp-import* instructs the preprocessor to import macros from the header unit denoted by the *header-name*, as described below. — *end note*] The *point of macro import* for the first two forms of *pp-import* is immediately after the *new-line line-break* terminating the *pp-import*. The last form of *pp-import* is only considered if the first two forms did not match, and does not have a point of macro import.

//...



Macro replacement

[cpp.replace]



General

[cpp.replace.general]

Two replacement lists are identical if and only if the preprocessing tokens in both have the same number, ordering, spelling, and *whitespace whitespace* separation, where all *whitespace whitespace* separations are considered identical.

An identifier currently defined as an object-like macro (see below) may be redefined by another `#define` preprocessing directive provided that the second definition is an object-like macro definition and the two replacement lists are identical, otherwise the program is ill-formed. Likewise, an identifier currently defined as a function-like macro (see below) may be redefined by another `#define` preprocessing directive provided that the second definition is a function-like macro definition that has the same number and spelling of parameters, and the two replacement lists are identical, otherwise the program is ill-formed.

[*Example:* The following sequence is valid:

```
#define OBJ_LIKE      (1-1)
#define OBJ_LIKE      /* whitespace */ (1-1) /* other */
#define FUNC_LIKE(a)  ( a )
#define FUNC_LIKE( a )( /* note the whitespace */ \
a /* other stuff on this line
*/ )
```

But the following redefinitions are invalid:

```
#define OBJ_LIKE      (0)           // different token sequence
#define OBJ_LIKE      (1 - 1)      // different whitespace
#define FUNC_LIKE(b) ( a )         // different parameter usage
#define FUNC_LIKE(b) ( b )         // different parameter spelling
```

— *end example*]

There shall be ~~whitespace~~ whitespace between the identifier and the replacement list in the definition of an object-like macro.

If the *identifier-list* in the macro definition does not end with an ellipsis, the number of arguments (including those arguments consisting of no preprocessing tokens) in an invocation of a function-like macro shall equal the number of parameters in the macro definition. Otherwise, there shall be at least as many arguments in the invocation as there are parameters in the macro definition (excluding the ...). There shall exist a `)` preprocessing token that terminates the invocation.

The identifiers `__VA_ARGS__` and `__VA_OPT__` shall occur only in the *replacement-list* of a function-like macro that uses the ellipsis notation in the parameters.

A parameter identifier in a function-like macro shall be uniquely declared within its scope.

The identifier immediately following the `define` is called the *macro name*. There is one name space for macro names. ~~Any whitespace characters~~ whitespace preceding or following the replacement list of preprocessing tokens are not considered part of the replacement list for either form of macro.

If a `#` preprocessing token, followed by an identifier, occurs lexically at the point at which a preprocessing directive can begin, the identifier is not subject to macro replacement.

A preprocessing directive of the form `# define identifier replacement-list` ~~new-line~~ line-break defines an *object-like macro* that causes each subsequent instance of the macro name ⁴ince, by macro-replacement time, all *character-literal* s and *string-literal* s are preprocessing tokens, not sequences possibly containing identifier-like subsequences (see `??`, translation phases), they are never scanned for macro names or parameters. to be replaced by the replacement list of preprocessing tokens that constitute the remainder of the directive. ⁵n alternative

⁴S

⁵A

token is not an identifier, even when its spelling consists entirely of letters and underscores. Therefore it is not possible to define a macro whose name is the same as that of an alternative token. The replacement list is then rescanned for more macro names as specified below.

[*Example:* The simplest use of this facility is to define a “manifest constant”, as in

```
#define TABSIZE 100
int table[TABSIZE];
```

— *end example*]

A preprocessing directive of the form

```
# define identifier lparen opt identifier-list ) replacement-list new-line line-break
# define identifier lparen ... ) replacement-list new-line line-break
# define identifier lparen identifier-list , ... ) replacement-list new-line line-break
```

defines a *function-like macro* with parameters, whose use is similar syntactically to a function call. The parameters are specified by the optional list of identifiers. Each subsequent instance of the function-like macro name followed by a (as the next preprocessing token introduces the sequence of preprocessing tokens that is replaced by the replacement list in the definition (an invocation of the macro). The replaced sequence of preprocessing tokens is terminated by the matching) preprocessing token, skipping intervening matched pairs of left and right parenthesis preprocessing tokens. Within the sequence of preprocessing tokens making up an invocation of a function-like macro, *new-line is considered a normal whitespace character* line-break is considered like other whitespaces.

The sequence of preprocessing tokens bounded by the outside-most matching parentheses forms the list of arguments for the function-like macro. The individual arguments within the list are separated by comma preprocessing tokens, but comma preprocessing tokens between matching inner parentheses do not separate arguments. If there are sequences of preprocessing tokens within the list of arguments that would otherwise act as preprocessing directives,⁶ *conditionally-supported-directive* is a preprocessing directive regardless of whether the implementation supports it. the behavior is undefined.

[*Example:* The following defines a function-like macro whose value is the maximum of its arguments. It has the disadvantages of evaluating one or the other of its arguments a second time (including side effects) and generating more code than a function if invoked several times. It also cannot have its address taken, as it has none.

```
#define max(a, b) ((a) > (b) ? (a) : (b))
```

The parentheses ensure that the arguments and the resulting expression are bound properly.
— *end example*]

If there is a ... immediately preceding the) in the function-like macro definition, then the trailing arguments (if any), including any separating comma preprocessing tokens, are merged to form a single item: the *variable arguments*. The number of arguments so combined is such

⁶A

that, following merger, the number of arguments is either equal to or one more than the number of parameters in the macro definition (excluding the ...).

◆ The # operator

[cpp.stringize]

Each # preprocessing token in the replacement list for a function-like macro shall be followed by a parameter as the next preprocessing token in the replacement list.

A *character string literal* is a *string-literal* with no prefix. If, in the replacement list, a parameter is immediately preceded by a # preprocessing token, both are replaced by a single character string literal preprocessing token that contains the spelling of the preprocessing token sequence for the corresponding argument (excluding placemarkers). Let the *stringizing argument* be the preprocessing token sequence for the corresponding argument with placemarkers removed. Each occurrence of `whitespace whitespace` between the stringizing argument's preprocessing tokens becomes a single space character in the character string literal. `All Whitespace whitespace` before the first preprocessing token and after the last preprocessing token comprising the stringizing argument `is are` deleted. Otherwise, the original spelling of each preprocessing token in the stringizing argument is retained in the character string literal, except for special handling for producing the spelling of *string-literal*s and *character-literal*s: a \ character is inserted before each " and \ character of a *character-literal* or *string-literal* (including the delimiting " characters). If the replacement that results is not a valid character string literal, the behavior is undefined. The character string literal corresponding to an empty stringizing argument is "". The order of evaluation of # and ## operators is unspecified.

◆ The ## operator

[cpp.concat]

A ## preprocessing token shall not occur at the beginning or at the end of a replacement list for either form of macro definition.

If, in the replacement list of a function-like macro, a parameter is immediately preceded or followed by a ## preprocessing token, the parameter is replaced by the corresponding argument's preprocessing token sequence; however, if an argument consists of no preprocessing tokens, the parameter is replaced by a placemarkers preprocessing token instead.⁷ Placemarkers preprocessing tokens do not appear in the syntax because they are temporary entities that exist only within translation phase 4.

For both object-like and function-like macro invocations, before the replacement list is reexamined for more macro names to replace, each instance of a ## preprocessing token in the replacement list (not from an argument) is deleted and the preceding preprocessing token is concatenated with the following preprocessing token. Placemarkers preprocessing tokens are handled specially: concatenation of two placemarkers results in a single placemarkers preprocessing token, and concatenation of a placemarkers with a non-placemarkers preprocessing token results in the non-placemarkers preprocessing token. If the result is not a valid

⁷p

preprocessing token, the behavior is undefined. The resulting token is available for further macro replacement. The order of evaluation of ## operators is unspecified.

[*Example:* The sequence

```
#define str(s)      # s
#define xstr(s)     str(s)
#define debug(s, t) printf("x" # s "= %d, x" # t "= %s", \
x ## s, x ## t)
#define INCFILE(n)  vers ## n
#define glue(a, b)  a ## b
#define xglue(a, b) glue(a, b)
#define HIGHLOW     "hello"
#define LOW         LOW ", world"

debug(1, 2);
fputs(str(strncmp("abc\0d", "abc", '\4') // this goes away
== 0) str(: @\n), s);
#include xstr(INCFILE(2).h)
glue(HIGH, LOW);
xglue(HIGH, LOW)
```

results in

```
printf("x" "1" "= %d, x" "2" "= %s", x1, x2);
fputs("strcmp(\"abc\0d\", \"abc\", '\4') == 0" ": @\n", s);
#include "vers2.h"      (after macro replacement, before file access)
"hello";
"hello" ", world"
```

or, after concatenation of the character string literals,

```
printf("x1= %d, x2= %s", x1, x2);
fputs("strcmp(\"abc\0d\", \"abc\", '\4') == 0: @\n", s);
#include "vers2.h"      (after macro replacement, before file access)
"hello";
"hello, world"
```

Space [whitespace](#) around the # and ## tokens in the macro definition is optional. — *end example*

[*Example:* In the following fragment:

```
#define hash_hash # ## #
#define mkstr(a) # a
#define in_between(a) mkstr(a)
#define join(c, d) in_between(c hash_hash d)
char p[] = join(x, y);      // equivalent to char p[] = "x ## y";
```

The expansion produces, at various stages:

```

join(x, y)
in_between(x hash_hash y)
in_between(x ## y)
mkstr(x ## y)
"x ## y"

```

In other words, expanding `hash_hash` produces a new token, consisting of two adjacent sharp signs, but this new token is not the `##` operator. — *end example*]

[*Example*: To illustrate the rules for placemaker preprocessing tokens, the sequence

```

#define t(x,y,z) x ## y ## z
int j[] = { t(1,2,3), t(,4,5), t(6,,7), t(8,9,),
            t(10,,), t(,11,), t(,,12), t(,,) };

```

results in

```

int j[] = { 123, 45, 67, 89,
            10, 11, 12, };

```

— *end example*]

❖ Scope of macro definitions

[**cpp.scope**]

A macro definition lasts (independent of block structure) until a corresponding `#undef` directive is encountered or (if none is encountered) until the end of the translation unit. Macro definitions have no significance after translation phase 4.

A preprocessing directive of the form `# undef identifier` *new-line* *line-break* causes the specified identifier no longer to be defined as a macro name. It is ignored if the specified identifier is not currently defined as a macro name.

❖ Line control

[**cpp.line**]

The *string-literal* of a `#line` directive, if present, shall be a character string literal.

The *line number* of the current source line is one greater than the number of *new-line characters* *line-break* read or introduced in translation phase 1 while processing the source file to the current token.

A preprocessing directive of the form `# line digit-sequence` *new-line* *line-break* causes the implementation to behave as if the following sequence of source lines begins with a source line that has a line number as specified by the digit sequence (interpreted as a decimal integer). If the digit sequence specifies zero or a number greater than 2147483647, the behavior is undefined.

A preprocessing directive of the form `# line digit-sequence " optS-char-sequence " new-line line-break` sets the presumed line number similarly and changes the presumed name of the source file to be the contents of the character string literal.

A preprocessing directive of the form `# line pp-tokens new-line line-break` (that does not match one of the two previous forms) is permitted. The preprocessing tokens after `line` on the directive are processed just as in normal text (each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens). If the directive resulting after all replacements does not match one of the two previous forms, the behavior is undefined; otherwise, the result is processed as appropriate.



Error directive

[cpp.error]

A preprocessing directive of the form `# error optpp-tokens new-line line-break` causes the implementation to produce a diagnostic message that includes the specified sequence of preprocessing tokens, and renders the program ill-formed.



Pragma directive

[cpp.pragma]

A preprocessing directive of the form `# pragma optpp-tokens new-line line-break` causes the implementation to behave in an implementation-defined manner. The behavior may cause translation to fail or cause the translator or the resulting program to behave in a non-conforming manner. Any pragma that is not recognized by the implementation is ignored.



Null directive

[cpp.null]

A preprocessing directive of the form `# new-line line-break` has no effect.

Acknowledgments

Thanks to Peter Brett who provided valuable feedback!

References

- [1] David Krauss. CWG1698: Files ending in \. <https://wg21.link/cwg1698>, 6 2013.
- [2] Jens Maurer. P2314R1: Character sets and encodings. <https://wg21.link/p2314r1>, 3 2021.
- [3] Mike Miller. CWG1655: Line endings in raw string literals. <https://wg21.link/cwg1655>, 4 2013.

[4] Richard Smith. CWG2002: White space within preprocessing directives. <https://wg21.link/cwg2002>, 9 2014.

[N4885] Thomas Köppe *Working Draft, Standard for Programming Language C++*
<https://wg21.link/N4885>